

Pixie Algorithm Explanation

Background and Origin

The Pixie algorithm originates from Pinterest's real-time recommendation engine, designed to recommend pins (images) to users in a highly dynamic and diverse environment. It was engineered to offer personalized, fast, and diverse recommendations even when traditional collaborative filtering techniques fall short.

Pixie relies on modeling the interaction data as a bipartite graph (users $\leftrightarrow$ items), and then simulates multiple biased random walks starting from a set of seed nodes (e.g., pins a user recently interacted with) to explore related content. In this project, we simulate a simplified version of Pixie tailored to movie recommendations.

Graph Structure and Construction

The first step in the Pixie-inspired algorithm is transforming the ratings data into a bipartite graph:

- **Nodes:**
 - User nodes (e.g., U1, U2)
 - Movie nodes (e.g., M101, M202)
- **Edges:**
 - Edges connect users to the movies they've rated.
 - Each edge can optionally be weighted by the rating (e.g., a 5-star rating could be a stronger edge).

Random Walk Process

The recommendation step involves random walks starting from a target user. The walker alternates between user nodes and movie nodes.

Algorithm Steps:

1. Start at a user node (the target user).
2. Randomly select a neighboring movie node.
3. From that movie, randomly choose a neighboring user.
4. Repeat steps 2–3 for a fixed number of steps or until a stopping condition.

5. Keep track of how often each movie is visited during the walk.
6. Rank the visited movies by frequency these become the recommendations.

Extensions and Enhancements

- **Biased Walks:** Prefer high-rated movies or frequent interactions.
- **Multiple Walks:** Run multiple walkers in parallel to improve coverage.
- **Early Stopping:** Stop if convergence is detected (top-k recommendations stabilize).

Limitations and Considerations

- **Performance Tuning:** The number of walks, walk length, and restart probabilities affect quality and performance.
- **Scalability:** Pure Python implementations may slow down at scale; production versions use C++, caching, and parallelism.
- **Repetition Bias:** Walkers may over-prefer popular movies unless de-biased.

How Pixie Can Be Extended

1. **Weighted Edges:** Use rating values or recency to bias walk probability.
2. **Restart Probability:** Similar to Personalized PageRank – restart at seed nodes.
3. **Node Embeddings:** Learn graph embeddings for better node similarity (e.g., Node2Vec).
4. **Hybrid Graphs:** Include user demographics, movie genres, or temporal signals as node features.